

main.c

```
1 #include <stdio.h>
2
3 void merge(int arr[], int left, int mid, int right) {
4     int n1 = mid - left + 1;
5     int n2 = right - mid;
6
7     int L[100000], R[100000];
8
9     for (int i = 0; i < n1; i++) L[i] = arr[left + i];
10    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
11
12    int i = 0, j = 0, k = left;
13    while (i < n1 && j < n2) {
14        if (L[i] <= R[j]) arr[k++] = L[i++];
15        else arr[k++] = R[j++];
16    }
17    while (i < n1) arr[k++] = L[i++];
18    while (j < n2) arr[k++] = R[j++];
19 }
20 void mergeSort(int arr[], int left, int right) {
21     if (left < right) {
22         int mid = (left + right) / 2;
23
24         mergeSort(arr, left, mid);
25         mergeSort(arr, mid + 1, right);
26         merge(arr, left, mid, right);
27     }
28 }
29 int main() {
30     int n;
31     printf("Enter number of students: ");
32     scanf("%d", &n);
33     int marks[100000];
34     printf("Enter %d marks: ", n);
35     for (int i = 0; i < n; i++) {
36         scanf("%d", &marks[i]);
37     }
38     mergeSort(marks, 0, n - 1);
39     printf("Sorted marks: ");
40     for (int i = 0; i < n; i++) {
41         printf("%d ", marks[i]);
42     }
43     printf("\n");
44     return 0;
45 }
```

Output

Enter number of students: 6
Enter 6 marks: 33 65 43 23 56 98
Sorted marks: 23 33 43 56 65 98

==> Code Execution Successful ==>

Clear

main.c

```
1 #include <stdio.h>
2
3 void merge(int arr[], int left, int mid, int right) {
4     int n1 = mid - left + 1;
5     int n2 = right - mid;
6     int L[100000], R[100000];
7     for (int i = 0; i < n1; i++) L[i] = arr[left + i];
8     for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
9     int i = 0, j = 0, k = left;
10    while (i < n1 && j < n2) {
11        if (L[i] <= R[j]) arr[k++] = L[i++];
12        else arr[k++] = R[j++];
13    }
14    while (i < n1) arr[k++] = L[i++];
15    while (j < n2) arr[k++] = R[j++];
16 }
17 void mergeSort(int arr[], int left, int right) {
18     if (left < right) {
19         int mid = (left + right) / 2;
20         mergeSort(arr, left, mid);
21         mergeSort(arr, mid + 1, right);
22         merge(arr, left, mid, right);
23     }
24 }
25 int main() {
26     int n;
27     printf("Enter number of products: ");
28     scanf("%d", &n);
29     int price[100000];
30     printf("Enter %d prices: ", n);
31     for (int i = 0; i < n; i++) {
32         scanf("%d", &price[i]);
33     }
34     mergeSort(price, 0, n - 1);
35     printf("Sorted prices: ");
36     for (int i = 0; i < n; i++) {
37         printf("%d ", price[i]);
38     }
39     printf("\n");
40     return 0;
41 }
```

Enter number of products: 5
Enter 5 prices: 23 55 76 99 11
Sorted prices: 11 23 55 76 99

=== Code Execution Successful ===

Clear

main.c

```
1 #include <stdio.h>
2
3 void merge(int arr[], int left, int mid, int right) {
4     int n1 = mid - left + 1;
5     int n2 = right - mid;
6     int L[100000], R[100000];
7     for (int i = 0; i < n1; i++) L[i] = arr[left + i];
8     for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
9     int i = 0, j = 0, k = left;
10    while (i < n1 && j < n2) {
11        if (L[i] <= R[j]) arr[k++] = L[i++];
12        else arr[k++] = R[j++];
13    }
14    while (i < n1) arr[k++] = L[i++];
15    while (j < n2) arr[k++] = R[j++];
16 }
17 void mergeSort(int arr[], int left, int right) {
18     if (left < right) {
19         int mid = (left + right) / 2;
20         mergeSort(arr, left, mid);
21         mergeSort(arr, mid + 1, right);
22         merge(arr, left, mid, right);
23     }
24 }
25 int main() {
26     int n;
27     printf("Enter number of employees: ");
28     scanf("%d", &n);
29     int salary[100000];
30     printf("Enter %d salaries: ", n);
31     for (int i = 0; i < n; i++) {
32         scanf("%d", &salary[i]);
33     }
34     mergeSort(salary, 0, n - 1);
35     printf("Sorted salaries: ");
36     for (int i = 0; i < n; i++) {
37         printf("%d ", salary[i]);
38     }
39     printf("\n");
40     int lowest = salary[0];
41     int highest = salary[n - 1];
42     int difference = highest - lowest;
43     printf("Lowest Salary: %d\n", lowest);
44     printf("Highest Salary: %d\n", highest);
45     printf("Difference: %d\n", difference);
46     return 0;
47 }
```

Output

```
Enter number of employees: 9
Enter 9 salaries: 768 46765534 57467 3456346 83456 247 873446 7974 134678
Sorted salaries: 247 764 7974 57467 83456 134678 873446 3456346 46765534
Lowest Salary: 247
Highest Salary: 46765534
Difference: 46765287
```

--- Code Execution Successful ---

Clear

main.c



Share

Run

Output

Clear

```

1 #include <stdio.h>
2
3 void swap(int *a, int *b) {
4     int temp = *a;
5     *a = *b;
6     *b = temp;
7 }
8 int partition(int arr[], int low, int high) {
9     int pivot = arr[high];
10    int i = low - 1;
11    for (int j = low; j < high; j++) {
12        if (arr[j] < pivot) {
13            i++;
14            swap(&arr[i], &arr[j]);
15        }
16    }
17    swap(&arr[i + 1], &arr[high]);
18    return i + 1;
19 }
20 void quickSort(int arr[], int low, int high) {
21    if (low < high) {
22        int pi = partition(arr, low, high);
23        quickSort(arr, low, pi - 1);
24        quickSort(arr, pi + 1, high);
25    }
26 }
27 int main() {
28    int n;
29    printf("Enter number of runners: ");
30    scanf("%d", &n);
31    int time[100000];
32    printf("Enter %d timings: ", n);
33    for (int i = 0; i < n; i++) {
34        scanf("%d", &time[i]);
35    }
36    quickSort(time, 0, n - 1);
37    printf("Sorted timings: ");
38    for (int i = 0; i < n; i++) {
39        printf("%d ", time[i]);
40    }
41    printf("\n");
42    return 0;
43 }
```

```

Enter number of runners: 6
Enter 6 timings: 6 5 2 9 8 4
Sorted timings: 2 4 5 6 8 9
```

⇒ Code Execution Successful ⇒

main.c

```
1 #include <stdio.h>
2
3 void swap(int *a, int *b) {
4     int temp = *a;
5     *a = *b;
6     *b = temp;
7 }
8 int partition(int arr[], int low, int high) {
9     int pivot = arr[high];
10    int i = low - 1;
11
12    for (int j = low; j < high; j++) {
13        if (arr[j] < pivot) {
14            i++;
15            swap(&arr[i], &arr[j]);
16        }
17    }
18    swap(&arr[i + 1], &arr[high]);
19    return i + 1;
20 }
21 void quickSort(int arr[], int low, int high) {
22    if (low < high) {
23        int pi = partition(arr, low, high);
24        quickSort(arr, low, pi - 1);
25        quickSort(arr, pi + 1, high);
26    }
27 }
28 int main() {
29    int n;
30    printf("Enter number of books: ");
31    scanf("%d", &n);
32    int id[100000];
33    printf("Enter %d book IDs: ", n);
34    for (int i = 0; i < n; i++) {
35        scanf("%d", &id[i]);
36    }
37    quickSort(id, 0, n - 1);
38    printf("Sorted IDs: ");
39    for (int i = 0; i < n; i++) {
40        printf("%d ", id[i]);
41    }
42    printf("\n");
43    return 0;
44 }
```

Output

Enter number of books: 4
Enter 4 book IDs: 6 44 88 76
Sorted IDs: 6 44 76 88

== Code Execution Successful ==

Clear

main.c

```
1 #include <stdio.h>
2
3 void swap(int *a, int *b) {
4     int temp = *a;
5     *a = *b;
6     *b = temp;
7 }
8 int partition(int arr[], int low, int high) {
9     int pivot = arr[high];
10    int i = low - 1;
11    for (int j = low; j < high; j++) {
12        if (arr[j] > pivot) {
13            i++;
14            swap(&arr[i], &arr[j]);
15        }
16    }
17    swap(&arr[i + 1], &arr[high]);
18    return i + 1;
19 }
20 void quickSort(int arr[], int low, int high) {
21    if (low < high) {
22        int pi = partition(arr, low, high);
23        quickSort(arr, low, pi - 1);
24        quickSort(arr, pi + 1, high);
25    }
26 }
27 int main() {
28     int n;
29     printf("Enter number of sellers: ");
30     scanf("%d", &n);
31     int sales[100000];
32     printf("Enter %d sales values: ", n);
33     for (int i = 0; i < n; i++) {
34         scanf("%d", &sales[i]);
35     }
36     quickSort(sales, 0, n - 1);
37     printf("Sorted sales (descending): ");
38     for (int i = 0; i < n; i++) {
39         printf("%d ", sales[i]);
40     }
41     printf("\n");
42     int sum = sales[0] + sales[1] + sales[2];
43     int average = sum / 3;
44     printf("Top 3: %d %d %d\n", sales[0], sales[1], sales[2]);
45     printf("Average: %d\n", average);
46     return 0;
47 }
```

Output

Enter number of sellers: 5
Enter 5 sales values: 42 36 98 76 45
Sorted sales (descending): 98 76 45 42 36
Top 3: 98 76 45
Average: 73

--- Code Execution Successful ---

Clear